

Fundamentos de PHP

introducción a las aplicaciones dinámicas



7

¿Qué es PHP?

PHP (acrónimo recursivo de PHP: *Hypertext Preprocessor*) es un lenguaje de programación de código abierto diseñado principalmente para el desarrollo web en el lado del servidor.

- **Lado del Servidor**, el código se ejecuta en el servidor, no en el navegador del usuario
- **Incrustable en HTML**, se puede mezclar directamente con código HTML
- **Código Abierto**, es gratuito y cuenta con una gran comunidad
- **Versatilidad**, es la base de sistemas populares como WordPress, Joomla y Magento

Fue creado por **Rasmus Lerdorf** (1994) como un conjunto de scripts en C/Perl llamados **Personal Home Page Tools (PHP Tools)**.

El objetivo inicial era mostrar visitas a una web personal y gestionar formularios simples.

8

Evolución de versiones



PHP/FI (1995–1997)

- *Forms Interpreter*
- Muy básico, sin estructura moderna

PHP 3 (1998)

- Primera versión **oficial**
- Introducción de extensiones
- Se convierte en lenguaje real de servidor

PHP 4 (2000)

- Motor Zend Engine 1.0
- Mejora de rendimiento

PHP 5 (2004)

- Programación orientada a objetos (POO) real
- Mejor soporte para: XML y MySQL
- Base del PHP moderno

PHP 7 (2015)

- Gran salto en rendimiento (x2)
- Tipado más fuerte
- manejo de errores moderno

PHP 8 (2020–actualidad)

- JIT (*Just-In-Time compilation*)
- Tipos estrictos mejorados
- match, atributos (*annotations modernas*) y union types

PHP en Linux VS PHP en Windows

Linux es el entorno más habitual en producción y Windows más común en desarrollo o contextos empresariales.

En sistemas Linux, PHP ofrece una **integración muy sólida** con servidores web como *Apache HTTP Server* y *Nginx*, lo que se traduce en **mayor estabilidad, rendimiento** y facilidad de despliegue en entornos reales.

En Windows, PHP suele utilizarse principalmente en entornos de desarrollo local o dentro de organizaciones que trabajan con tecnología Microsoft, donde se integra con IIS.

Aunque el rendimiento y la estabilidad han mejorado con los años, su uso en producción es menos frecuente.



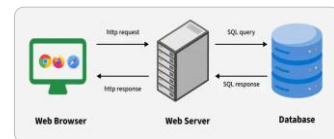
Side Server

PHP trabaja en el servidor.

1. el navegador del cliente envía una petición
2. el servidor ejecuta código en PHP y devuelve una respuesta en HTML (o JSON)

Característica	PHP / ASP.NET Core / Java	Angular / React
Ejecución	En el servidor	En el navegador
Lenguajes	PHP, C#, Java	JavaScript / TypeScript
Acceso a BD	Sí (ej: MySQL)	No directo
Generación de HTML	Sí (dinámico)	Sí (renderizado en cliente)
Seguridad	Alta (control servidor)	Depende del backend
Ejemplos	ASP.NET Core, Spring	Angular, React
Comunicación	Expone APIs	Consume APIs

```
PHP
<?php
echo "<h1>Hola desde PHP</h1>";
?>
```



14

Side Client

El navegador del usuario representa el *Side Client*, donde únicamente se trabaja con tres tecnologías:

- **HTML** → estructura, se utiliza para definir el contenido de la página
- **CSS** → presentación, controla el diseño, la apariencia (colores, tamaños, layout, ...)
- **JavaScript** → comportamiento, añadiendo interactividad a las páginas

Hoy en día, **JavaScript es imprescindible** porque:

- Permite aplicaciones dinámicas sin recargar la página (SPA)
- Gestiona eventos como clicks, envío de formularios, animaciones, ...
- Consume APIs (fetch / AJAX)

Sin JavaScript, la web sería prácticamente estática.

PHP trabaja en el servidor, mientras JavaScript lo hace en el cliente.



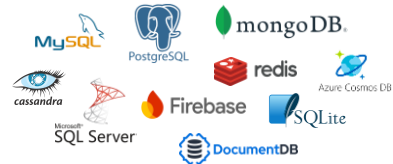
15

Bases de datos en aplicaciones PHP

En webs o APIs desarrolladas con PHP, **el acceso a la base de datos se realiza siempre desde el servidor**, nunca directamente desde el cliente.

El acceso desde el servidor es **clave para la seguridad y el control de la lógica de negocio**:

- el navegador solicita datos
- PHP procesa la petición
- accede a la base de datos
- devuelve la información (normalmente en JSON o HTML)



PHP proporciona extensiones para interactuar con bases de datos.

Una **extensión es un módulo o librería que amplía las capacidades de PHP**, permitiendo añadir funcionalidades específicas (por ejemplo, conectarse a bases de datos, trabajar con XML, gestionar imágenes, etc.).

16

Creación de APIs con PHP

PHP tiene una opción básica es trabajar directamente con variables globales com:

- `$_SERVER['REQUEST_URI']`
permite conocer la ruta solicitada por el cliente, por ejemplo /users o /products/5
- `$_SERVER['REQUEST_METHOD']`
indica el método HTTP usado en la petición, como GET, POST, PUT o DELETE

Con ambas variables, en código PHP puro, se puede construir un enrutado manual:

- se analiza la URL recibida
- se comprueba el método
- se decide qué lógica ejecutar

En proyectos reales puede volverse difícil de mantener a medida que crecen las rutas, las validaciones y la lógica de negocio.



17

Creación de APIs con PHP + Laravel



Frameworks como **Laravel simplifican mucho este trabajo** porque añaden:

- un sistema de routing declarativo
- un ORM llamado Eloquent para trabajar con la base de datos mediante modelos
- middleware para interceptar peticiones y aplicar reglas como:
 - autenticación
 - control de acceso
 - mecanismos de validación para verificar los datos recibidos antes de procesarlos

PHP

```
Route::get('/users', function () {  
    return User::all();  
});
```

Laravel ya interpreta la ruta, restringe el método HTTP a GET y devuelve todos los registros del modelo User, normalmente serializados como JSON.

Esto reduce mucho el código repetitivo y hace que la API sea más legible y mantenible.

18

Creación de APIs con PHP + Symfony



En **Symfony** el enfoque es similar, aunque **más orientado a controladores y atributos**.

En lugar de definir rutas con funciones anónimas, se suelen declarar sobre métodos de una clase controladora usando atributos como `#[Route('/users', methods: ['GET'])]`.

PHP

```
#[Route('/users', methods: ['GET'])]  
public function getUsers(): JsonResponse {  
    return $this->json($users);  
}
```

Los métodos pueden devolver una `JsonResponse`, por ejemplo:
`return $this->json($users);`

Symfony también incorpora routing, validación, middleware a través del sistema de eventos y firewalls, además de integración con ORM mediante Doctrine.

19

Creación de APIs con JS

Combinar PHP con un backend en Node.js para exponer APIs es una arquitectura cada vez más común, pero no siempre es la mejor opción.

✓ Ventajas

- Asincronía nativa → ideal para APIs de alto tráfico o un I/O intensivo (APIs externas, colas, sockets)
- Tiempo real (WebSockets) sencillo (Socket.io)
- Ecosistema JS unificado (frontend + backend)
- Muy bueno para microservicios

✗ Desventajas

- Complejidad añadida (**dos stacks distintos**)
- Gestión de errores más compleja (async/await)
- Menos estructurado por defecto (vs Symfony/Laravel)
- Necesitamos gestionar, npm, seguridad y arquitectura adicional



20

CRUD en una API

Cuando construyes una API con PHP, el patrón fundamental es CRUD, que representa las operaciones básicas sobre datos y se mapea directamente con métodos HTTP.



Operación	Método HTTP	Ejemplo URL	Descripción
Create	POST	/api/users	Crear un recurso
Read	GET	/api/users	Obtener lista
Read	GET	/api/users/1	Obtener uno
Update	PUT / PATCH	/api/users/1	Actualizar
Delete	DELETE	/api/users/1	Eliminar

21

Comunicación HTTP en una API

HTTP Headers (Cabeceras del mensaje)

Las *cabeceras* proporcionan metadatos sobre la petición o respuesta.

- **Content-Type** → formato del cuerpo enviado
- **Accept** → formato esperado en respuesta
- **Authorization** → autenticación

```
HTTP
Content-Type: application/json
Authorization: Bearer TOKEN
Accept: application/json
```

22

Comunicación HTTP en una API

HTTP Body (Cuerpo del mensaje)

El *body* contiene los datos enviados al servidor o el cliente (principalmente en POST/PUT/PATCH).

El formato de los datos se indica mediante la cabecera *Content-Type*:

- **text/plain** → formato en texto plano
- **application/json** → formato JSON
- **application/xml** → formato XML
- **application/x-www-form-urlencoded** → procesamiento de formularios
- **multipart/form-data** → subida de archivos

23

Comunicación HTTP en una API

HTTP Status Codes

Los códigos de estado indican el resultado de la operación:

Código	Significado	Uso típico
200	OK	Respuesta correcta
201	Created	Recurso creado
204	No Content	Sin contenido
400	Bad Request	Error cliente
401	Unauthorized	No autenticado
403	Forbidden	Sin permisos
404	Not Found	No existe
500	Server Error	Error servidor



24

Comunicación HTTP en una API

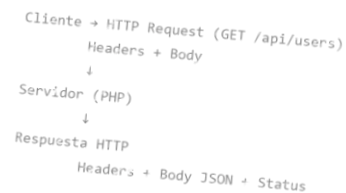
Formato de las URLs

Las URLs deben ser:

- ☒ Usar sustantivos → /users
- ☒ No usar verbos → /getUsers
- ☒ Usar plural → /users
- ☒ Versionar API → /api/v1/users
- ☒ Jerarquía clara → /users/1/posts

Ejemplo

```
/api/users  
/api/users/1  
/api/users/1/orders
```



25

Herramientas para el desarrollo

XAMPP es un entorno de desarrollo local que integra en un solo paquete todo lo necesario para trabajar con PHP:

- servidor web (*Apache*)
- base de datos (*MySQL/MariaDB*)
- herramientas como *phpMyAdmin*

Está pensado especialmente para facilitar el inicio en desarrollo web, evitando configuraciones complejas, y permite levantar rápidamente un entorno funcional en Windows, Linux o macOS para probar aplicaciones y APIs de forma local.



26

Instalación de Visual Studio Code

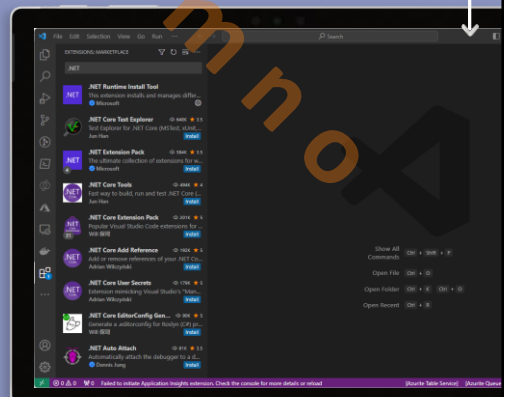
1. Descargamos e instalamos Visual Studio Code.
2. Finalizada la instalación instalamos las extensiones que añaden características al entorno de desarrollo.

Las extensiones de *VSCode* son paquetes de código que se ejecutan dentro del entorno de desarrollo.

<https://visualstudio.microsoft.com/es/>

Las extensiones que necesitamos para proyectos PHP

- PHP Intelephense → autocompletado y análisis
- PHP Debug (con Xdebug) → debugging real
- Prettier
- ESLint
- Auto Rename Tag
- vscode-icons (recomendó)



27

Instalación de XAMPP

1. Descargar desde la web oficial de XAMPP
2. Ejecutar instalador
3. Seleccionar componentes, Apache, MySQL y PHP
4. Instalar en C:\xampp
5. Abrir panel de control XAMPP
6. Iniciar Apache y MySQL

C:\xampp\htdocs\ es la carpeta principal de tus proyectos.

htdocs/
mi-proyecto/
index.php

Acceso desde navegador:
`http://localhost/mi-proyecto`

<https://www.apachefriends.org/es/index.html>

El panel de control de XAMPP es desde donde se gestionan todos los servicios.

- **Start:** Iniciar un servicio (Apache, MySQL, etc.)
- **Stop:** Detener un servicio en ejecución
- **Restart:** Reiniciar el servicio
- **Config:** Acceder a archivos de configuración
- **Logs:** Ver registros de errores y actividad
- **Admin:** Abrir herramientas en el navegador
- **Netstat:** Ver puertos en uso y detectar conflictos

